

흉부 수술 환자 임상 정보 기반 1년 내 사망 여부 예측 딥러닝 프로젝트

공개 의료 표 데이터 기반 이진 분류 미니 프로젝트

개인 학습형 프로젝트

Contact

—

010.9342.9357

khs0808sky@naver.com

Developer

—

Kim hyunsu (1996.08.08)

Kim hyun su Portfolio

Contents

1 프로젝트 목표 및 흐름

2 데이터 소개

3 전처리 과정

4 모델 설계

5 학습 및 평가

6 결과와 한계

7 느낀 점

프로젝트 목표 및 흐름

프로젝트 목표

환자 임상 정보 16개를 입력값으로 사용

수술 후 1년 사망 여부를 0/1로 예측

기본 전처리, 학습, 평가, 예측 과정을 직접 구현

1

전체 진행 흐름

데이터 수집 → 데이터 구조 확인

→ 전처리 → 모델 설계 → 학습

→ 평가 → 예측 결과 확인

2

사용 환경 및 기술

Python

TensorFlow / Keras

Google Colab

pandas, Numpy

3

데이터 소개

데이터 출처: UCI Machine Learning Repository, Thoracic Surgery Data (Dataset ID 277)

데이터 주제: 흉부 수술 환자의 1년 내 사망 여부 예측

데이터 규모: 470개 사례, 입력 특성 16개

문제 유형: 이진 분류

데이터 특징: 의료 표 데이터, 클래스 불균형 존재

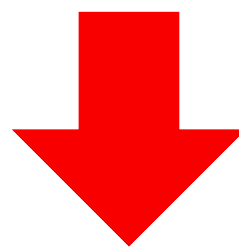
DGN	object	DGN: 진단 코드 조합
PRE4	float64	PRE4: 폐활량 FVC
PRE5	float64	PRE5: 1초간 강하게 내쉰 공기량 FEV1
PRE6	object	PRE6: 수술 전 전신 상태 점수(Zubrod scale)
PRE7	object	PRE7: 수술 전 통증 여부
PRE8	object	PRE8: 수술 전 객혈 여부
PRE9	object	PRE9: 수술 전 호흡곤란 여부
PRE10	object	PRE10: 수술 전 기침 여부
PRE11	object	PRE11: 수술 전 쇠약감 여부
PRE14	object	PRE14: 종양 크기 관련 임상 T 단계
PRE17	object	PRE17: 제2형 당뇨 여부
PRE19	object	PRE19: 최근 6개월 내 심근경색 여부
PRE25	object	PRE25: 말초동맥질환 여부
PRE30	object	PRE30: 흡연 여부
PRE32	object	PRE32: 천식 여부
AGE	float64	AGE: 수술 당시 나이
Risk1Yr	object	Risk1Yr: 1년 내 사망 여부

전처리 과정

```
1 from scipy.io import arff
2 import pandas as pd
3
4 # 1. arff 파일 읽기
5 data, meta = arff.loadarff("ThoracicSurgery.arff")
6
7 # 2. pandas DataFrame으로 변환
8 df = pd.DataFrame(data)
9
10 # 3. 바이트 문자열이 있으면 일반 문자열로 바꾸기
11 for col in df.columns:
12     if df[col].dtype == object:
13         df[col] = df[col].apply(lambda x: x.decode("utf-8") if isinstance(x, bytes) else x)
14
15 # 4. csv로 저장
16 df.to_csv("thoracic_surgery_data.csv", index=False, encoding="utf-8-sig")
17
18 # 5. 확인
19 print(df.head())
20 print(df.shape)
21 print(df.dtypes)
```

 ThoracicSurgery.arff

※ .arff파일을 .csv파일로 변환하는 코드



 thoracic_surgery_data.csv

```
1 import pandas as pd
2
3 df = pd.read_csv("thoracic_surgery_data.csv")
```

```
1 X = df.drop("Risk1Yr", axis=1)
2 y = df["Risk1Yr"]
```

X : 입력 특성 16개

Y : 1년 내 사망 여부

전처리 과정

타깃 라벨을 숫자형(0/1)으로 변환

```
y = y.map({"F": 0, "T": 1})
```

범주형 값을 숫자형으로 변환

```
# 1) DGN: DGN2 -> 1, DGN3 -> 2, ... (뒤 숫자 추출 후 -1)
X["DGN"] = X["DGN"].str.replace("DGN", "", regex=False).astype(int) - 1

# 2) PRE4, PRE5는 원래 숫자라 그대로 둠

# 3) PRE6: PRZ0 -> 0, PRZ1 -> 1, PRZ2 -> 2
X["PRE6"] = X["PRE6"].str.replace("PRZ", "", regex=False).astype(int)

# 4) PRE7 ~ PRE11: F -> 0, T -> 1
binary_cols_1 = ["PRE7", "PRE8", "PRE9", "PRE10", "PRE11"]
for col in binary_cols_1:
    X[col] = X[col].map({"F": 0, "T": 1})

# 5) PRE14: OC11 -> 0, OC12 -> 1, OC13 -> 2, OC14 -> 3
X["PRE14"] = X["PRE14"].str.replace("OC", "", regex=False).astype(int) - 11

# 6) PRE17, PRE19, PRE25, PRE30, PRE32: F -> 0, T -> 1
binary_cols_2 = ["PRE17", "PRE19", "PRE25", "PRE30", "PRE32"]
for col in binary_cols_2:
    X[col] = X[col].map({"F": 0, "T": 1})

# 7) AGE는 원래 숫자라 그대로 둠
```

전처리한 데이터를 CSV 파일로 저장

```
processed_df = X.copy()
processed_df["Risk1Yr"] = y

processed_df.to_csv("thoracic_surgery_processed.csv", index=False, encoding="utf-8-sig")
```

학습용 / 테스트용 데이터 분리

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

모델 설계

입력층: 환자 임상 정보 16개

은닉층: Dense(30), 활성화 함수 ReLU

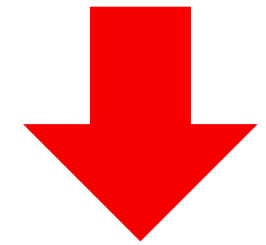
출력층: Dense(1), 활성화 함수 sigmoid

문제 유형: 이진 분류

```
model = Sequential()  
model.add(Dense(30, input_shape=(16,), activation='relu'))  
model.add(Dense(1, activation='sigmoid'))
```

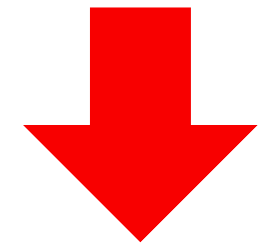
Input Layer

16 features



Hidden Layer

Dense(30), ReLU



Output Layer

Dense(1), sigmoid

학습 및 평가

```
model.compile(  
    loss='binary_crossentropy',  
    optimizer='adam',  
    metrics=['accuracy']  
)
```

손실 함수: binary_crossentropy

최적화 함수: adam

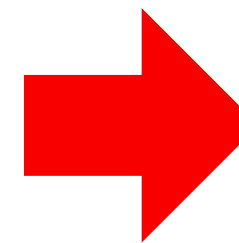
평가 지표: accuracy

```
history = model.fit(  
    X_train, y_train,  
    epochs=30,  
    batch_size=16,  
    validation_split=0.2,  
    verbose=1  
)
```

학습 횟수: epochs=30

배치 크기: batch_size=16

```
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)  
print("테스트 손실값:", loss)  
print("테스트 정확도:", accuracy)
```



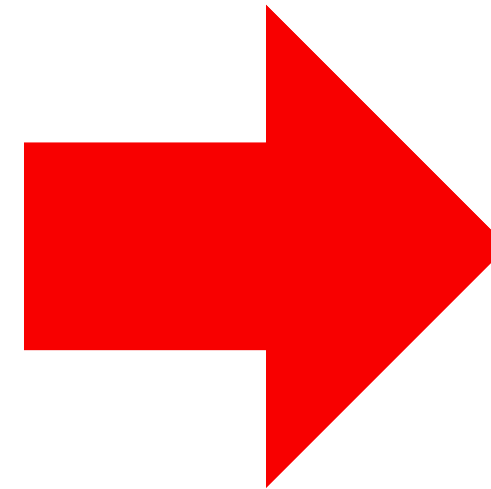
```
테스트 손실값: 0.43065890669822693  
테스트 정확도: 0.8510638475418091
```


예측 예시

```
sample = X_test.iloc[[0]]
pred_prob = model.predict(sample, verbose=0)
pred_label = 1 if pred_prob[0][0] >= 0.5 else 0
actual_label = y_test.iloc[0]

print("샘플 환자 입력값:")
print(sample.T)

print("\n예측 확률:", pred_prob[0][0])
print("예측 라벨:", pred_label)
print("실제 라벨:", actual_label)
```



테스트 데이터 샘플 1건

```
샘플 환자 입력값:
      374
DGN      2.00
PRE4      2.40
PRE5      2.16
PRE6      1.00
PRE7      0.00
PRE8      0.00
PRE9      0.00
PRE10     1.00
PRE11     0.00
PRE14     1.00
PRE17     0.00
PRE19     0.00
PRE25     0.00
PRE30     1.00
PRE32     0.00
AGE      69.00

예측 확률: 0.14594069
예측 라벨: 0
실제 라벨: 0
```

모델은 해당 샘플의 1년 내 사망 확률을 0.1459로 예측

0.5 기준에 따라 최종 라벨을 0으로 분류

실제 라벨도 0으로 확인되어 예측이 일치

결과와 한계

결과

테스트 데이터 기준 정확도는
약 85.1%로 확인됨

샘플 환자 1건에 대한
예측 결과에서 예측 라벨과 실제 라벨이
일치하는 사례를 확인

공개 의료 표 데이터를 활용해
이진 분류 모델의 전체 흐름을
직접 구현했다는 점에 의미가 있음

한계

데이터 규모가 크지 않아
결과 해석에 주의가 필요

클래스 불균형이 있어
정확도만으로 성능을 판단하기 어려움

향후 평가 지표 확장과
모델 구조 개선이 가능

느낀 점

1 AI의 도움을 받아가며 직접 실습해보니, 단순히 내용을 읽는 것보다 실제로 공부하고 있다는 느낌을 받을 수 있었다.

2 딥러닝 전체를 깊게 이해한 단계는 아니지만, 데이터 전처리부터 모델 설계, 학습, 예측까지의 기본 흐름을 직접 경험할 수 있었다.

3 공개 의료 표 데이터를 활용해 하나의 프로젝트 형태로 끝까지 정리해보면서, 개념이 조금 더 구체적으로 잡히는 경험을 했다.

4 앞으로는 평가 지표를 더 다양하게 확인하고, 모델 구조를 비교해보는 방식으로 확장해보고 싶다.



Thank you

Contact

010.9342.9357
khs0808sky@naver.com

Developer

Kim hyunsu (1996.08.08)